

Experimento 4

Circuitos Combinacionais: Comparador de Palavras

Grupo B2 - Turma T2
Allan Victor Almeida Faria, 261010546
Caio César Gonçalves Ribeiro, 211038217
Danilo Alves Ribeiro, 242003754

¹Dep. Ciência da Computação – Universidade de Brasília (UnB)
CIC0231 - Laboratório de Circuitos Lógicos
2 de outubro de 2026

allanvictor.almeida@gmail.com, caiocesar20032003@gmail.com, daniloalvesjob@gmail.com

Abstract. *This experiment aimed to develop and simulate digital comparator circuits using the Quartus II software. The circuits were constructed from native logic components and reusable sub-circuits, applying concepts such as modularity, reuse, and encapsulation to the development of electronic systems. The designed circuits were simulated to verify their logical behavior and operation according to their respective truth tables. The results demonstrated the applicability of software-based electronic design tools in the development and verification of digital circuits.*

Resumo. *Este experimento teve como objetivo desenvolver e simular circuitos comparadores digitais utilizando o software Quartus II. Os circuitos foram construídos a partir de componentes lógicos nativos e subcircuitos reutilizáveis, aplicando conceitos como modularidade, reuso e encapsulamento ao desenvolvimento de sistemas eletrônicos. Os circuitos projetados foram simulados para verificar seu comportamento lógico e funcionamento de acordo com suas respectivas tabelas verdade. Os resultados demonstraram a aplicabilidade de ferramentas computacionais de projeto eletrônico no desenvolvimento e na verificação de circuitos digitais.*

1. Introdução

O uso de ferramentas de software revolucionou o desenvolvimento de projetos eletrônicos. Sintetizadores, simuladores e softwares de esquemáticos permitem a produção e teste de circuitos em larga escala, além de democratizar o acesso a estudantes e pequenos desenvolvedores.

Nesse sentido, princípios de desenvolvimento de software, como reuso e encapsulamento são aplicados na prototipação de circuitos eletrônicos, facilitando e agilizando tal processo. Assim, foram desenvolvido e simulados no Quartus II circuitos comparadores, utilizando-se de sub-circuitos reutilizáveis construídos a partir de circuitos nativos.

1.1. Objetivos

Implementar e simular, no Quartus II, comparadores com entradas vetoriais.

1.2. Materiais

Neste experimento foram utilizados os seguintes materiais e equipamentos:

- Simulador Quartus II

2. Procedimentos e Resultados

2.1. Comparador de 3 bits de Igualdade

Para fazer tal comparador, devemos comparar bit-a-bit, em uma função ideal que retorne '1' se e somente se os dois bits comparados forem iguais. Após isso, basta submeter as saídas desses operadores a uma AND de três portas.

O operador lógico binário cuja saída é 1 apenas quando as entradas são iguais é o XNOR[2].

Tabela 1. Tabela Verdade da função XNOR

A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Dessa forma, tal expressão equivale, em mintermos, a:

$$S = AB + \bar{A}.\bar{B} \quad (1)$$

Em NAND, tem-se:

$$Z = \overline{\overline{A}\overline{A}\overline{B}\overline{B}\overline{A}\overline{B}}$$

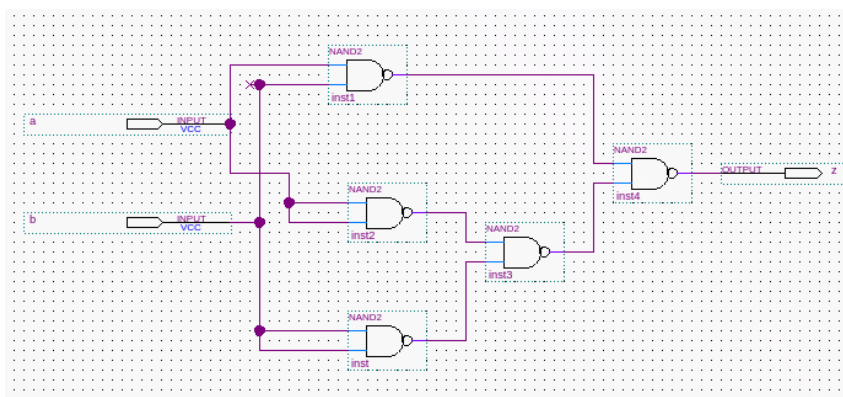


Figura 1. NXOR com NAND

A confecção de uma NAND de três, implementada no Experimento 3, resulta no circuito:

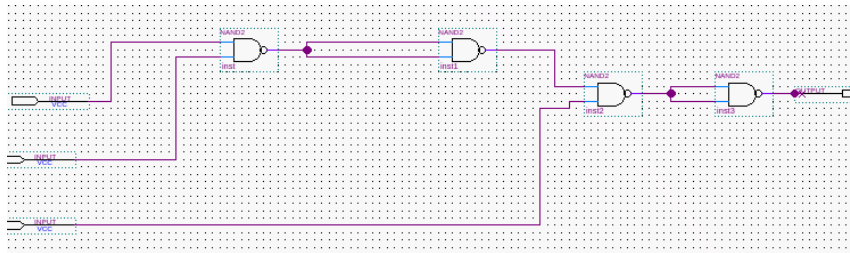


Figura 2. NAND3

Dessa forma, nosso circuito completo será:

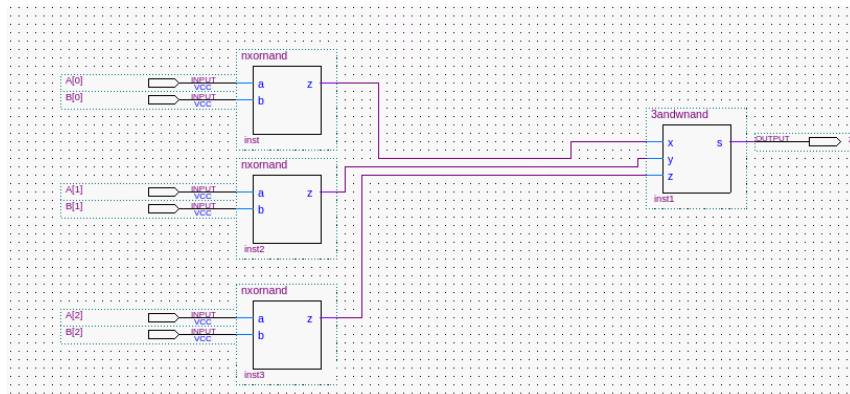


Figura 3. Circuito Completo

Realizando a simulação funcional, temos:

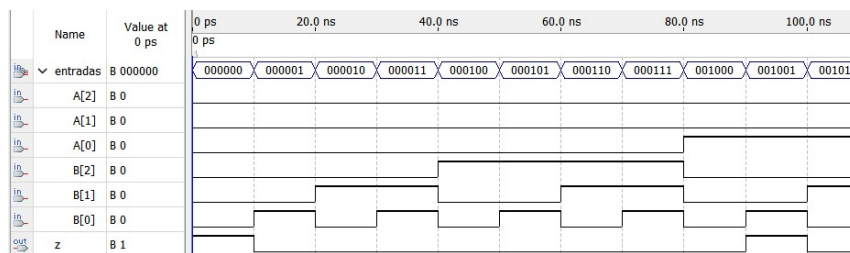


Figura 4. Simulação Funcional

Percebe-se que a saída só é '1' quando os vetores são iguais, comprovando a validade do modelo. Em relação à análise temporal, a qual leva em conta os atrasos das portas lógicas, temos:

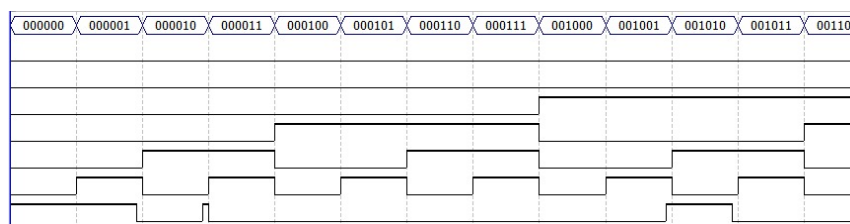


Figura 5. Simulação Temporal

2.2. Comparador de Palavras de 2 Bits

Além da noção de igualdade, agora teremos saídas para verificar se um número é maior ou menor a outro, seguindo uma implementação modular, na qual o comparador vetorial é construído a partir do comparador de bits individuais.

Em um sistema binário, para um bit ser maior que outro, basta que este seja '0' e aquele seja '1'. O raciocínio é análogo para o caso de "menor que", logo:

$$y1 = A\overline{B}$$
(2)

$$y2 = \overline{A \oplus B}$$
(3)

$$y3 = \overline{A}B$$
(4)

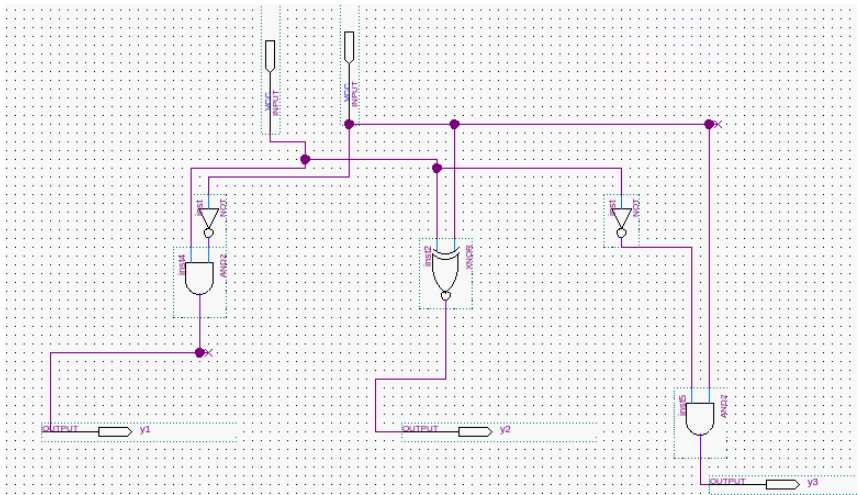


Figura 6. Comparador de 1 Bit

A simulação funcional do circuito nos dá:

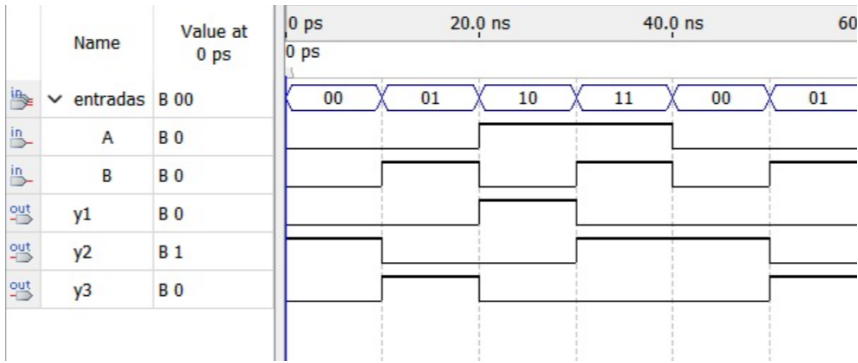


Figura 7. Simulação Funcional

Já a simulação temporal:

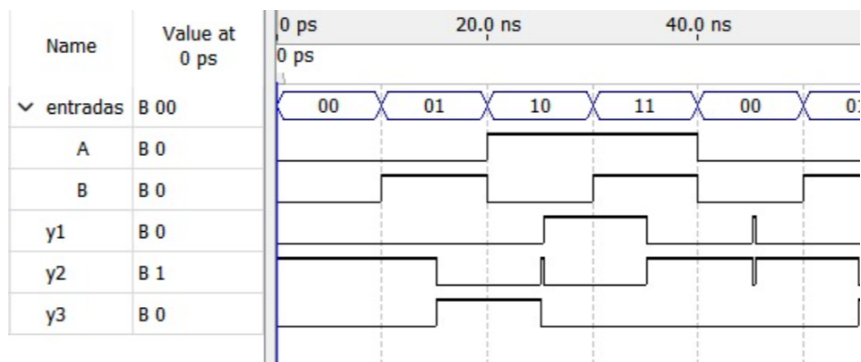


Figura 8. Simulação Temporal

No circuito final, sempre são testados os bits mais significativos. Caso estes sejam iguais, o circuito dos bits menos significativos é habilitado. Seguindo o circuito disponibilizado no roteiro[1], tem-se:

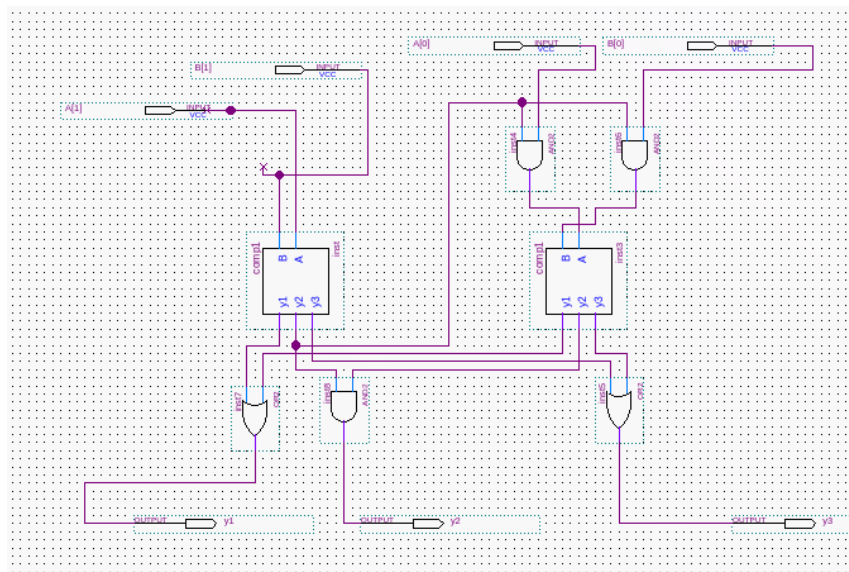


Figura 9. Comparador de 2 Bits

3. Conclusões

Os experimentos conduzidos confirmaram a teoria. Nesse sentido, a distinção entre simulação funcional e temporal ficou clara ao diagnosticar os atrasos desta última na simulação do segundo comparador. De maneira geral, o uso de ferramentas de software provou-se importantíssimo para construção e verificação de circuitos mais complexos.

Referências

- [1] Roteiro. https://aprender3.unb.br/pluginfile.php/3416792/mod_resource/content/8/roteiro4P.pdf, 2026. [Online; accessed 2-October-2026].

[2] John F. Wakerly. *Digital Design: Principles and Practices*. Pearson, 3rd edition, 1990.

Auto-Avaliação

1. b
2. d
3. d. O circuito é impossível apenas com XOR
4. d
5. Se a constante '1' está disponível, "a". Caso contrário, "d"